KUL

USER GUIDE

ORFS Flow Runner

Install it, run a design from RTL to GDS and read the results, shown on real CVA6 runs, including four stock and parallel abc_new synthesis runs

AUTHOR	rafay
DATE	2026-11-09
APP VERSION	1.1.0
DESIGN AND PLATFORM	cva6 on asap7, hierarchical
MACHINE	Intel Core i5-1235U host, VirtualBox VM with 8 vCPUs and 10 GB RAM, Ubuntu 24.04

Contents

1. Summary **3**

2. At a glance **3**

3. Installing **4**

3.1 Build and install the package 4
3.2 First start 4
3.3 Choose the tools 4

4. The window **6**

5. Running a design **7**

6. Setting the inputs **10**

6.1 Clocks 10
6.2 Constraints 12
6.3 IO pins 14
6.4 Flow settings 14
6.5 Files and new configs 15
6.6 Saving 16

7. Reading the results **16**

7.1 Reports → QoR checks 16
7.2 Reports → Report files 17
7.3 Logs 17
7.4 Debug 18

8. Worked example: stock and parallel abc_new **19**

8.1 What was compared 19
8.2 How to repeat it 20
8.3 Results 20
8.4 What the numbers mean 21

9. Other tasks **22**

9.1 Re-run from a stage 22
9.2 Runs that must outlive the app 23
9.3 Open a stage in the OpenROAD GUI 23

10. Troubleshooting **23**

11. Files **24**

1. Summary

ORFS Flow Runner is a desktop app for OpenROAD-flow-scripts (ORFS). It runs any design in an ORFS flow folder from RTL to GDS through the stock ORFS Makefile, so a run in the app gives the same result as the same make command in a terminal. This guide shows every part of the app with screenshots taken from real runs on this machine.

- **Install:** one Debian package, `orfs-flow-runner_1.1.0_all.deb`, puts the app under /opt with a menu entry. OpenROAD, Yosys and ORFS are installed separately; the app finds them on PATH or where Tools... points.
- **Set the inputs:** the Inputs tab edits what a run reads, without a terminal: clocks found in the top module's RTL with their periods and relationships, the SDC, where the IO pins go, every ORFS flow variable and the design's files. Saves keep a backup and point From stage at the first stage the change affects (section 6).
- **Run:** pick platform, design, config, CPU cores and a stage range from drop-down menus. The plan box shows the exact make command before anything runs.
- **Read results:** Reports shows pass/fail tiles and a metric-by-stage table; Logs and Report files have a line filter; Debug shows the environment, the resolved make variables and every warning ID.
- **Worked example:** CVA6 (asap7, hierarchical) synthesised four times from the app: stock Yosys, parallel abc_new, parallel abc_new, stock Yosys. The netlists are identical in all four. abc_new was 1.29× faster in the cleanest pair of runs, but the machine slowed down during the later runs, and in that pair the parallel abc_new was slower (0.85×). Averaged over two runs per build, abc_new is 1.02× and the whole synthesis 1.12× faster. Run-to-run noise on this machine is as large as the effect, so section 8 reports the range, not one number.

2. At a glance

Item	Value
App version	1.1.0 (gui/orfs_app.py)
Package	orfs-flow-runner_1.1.0_all.deb, built by gui/packaging/build_deb.sh
Needs	Python 3, GTK 3 with PyGObject, make, tcl (to read clocks from an SDC); KLayout for the final GDS step
Installed separately	OpenROAD, Yosys, an OpenROAD-flow-scripts checkout
Tested on	Ubuntu 24.04 in a VirtualBox VM: Intel Core i5-1235U host, 8 vCPUs, 10 GB RAM
Tool versions in the screenshots	OpenROAD 26Q4-271-g0907a8656e; Yosys 0.69+274 (stock) and 0.69+280 (parallel abc_new); KLayout 0.28.16
Design in the screenshots	cva6 on asap7, hierarchical synthesis (config_hier*.mk); section 6: cva6 on nangate45, and ORFS's chi_async_bridge and ethmac on asap7
Per-user files	~/.config/orfs-flow-runner, ~/.local/share/orfs-flow-runner/run_logs, ~/.cache/orfs-flow-runner

3. Installing

3.1 Build and install the package

From a checkout of KUL, build the package and install it with apt. apt also installs GTK, PyGObject, make and KLayout when they are missing.

```
gui/packaging/build_deb.sh
sudo apt install ./dist/orfs-flow-runner_1.0.1_all.deb
```

The package installs the app in /opt/orfs-flow-runner, adds ORFS Flow Runner to the app menu and adds an orfs-flow-runner command. To update, build and install the newer package the same way. sudo apt remove orfs-flow-runner removes it and keeps your settings and run logs.

If you used gui/install_launcher.sh before, run gui/install_launcher.sh --uninstall first: its menu entry has the same name and would hide the installed one.

3.2 First start

Start ORFS Flow Runner from the app menu. It opens the flow folder you used last, else $ORFS_FLOW, else ~/OpenROAD-flow-scripts/flow. If it finds none it asks for one; pick the flow/ folder of your ORFS checkout. Flow folder… in the title bar changes it later.

3.3 Choose the tools

Tools… in the title bar sets which OpenROAD, Yosys and KLayout programs the runs use. Leave a row on Automatic to use the program on PATH, or the one in /usr/local/bin, /usr/bin, /opt/openroad/bin or ~/.local/bin. An app started from the menu has a short PATH, so this is how it finds tools installed elsewhere. Choices are saved in ~/.config/orfs-flow-runner/settings.json.

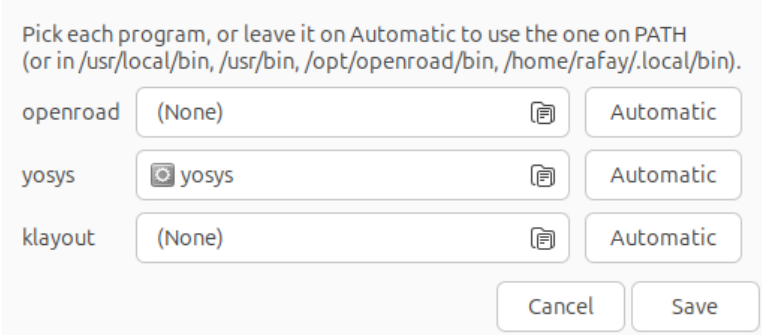


Figure 1. Tools… with Yosys set to the parallel abc_new build (yosys/build/yosys); OpenROAD and KLayout stay on Automatic.

Debug → Environment then shows the version and path of each tool, whether Yosys has the read_slang frontend that CVA6 needs, the CPU count, free memory and the run-log folder. Check it once after installing: a NOT FOUND row means a run would fail at once.

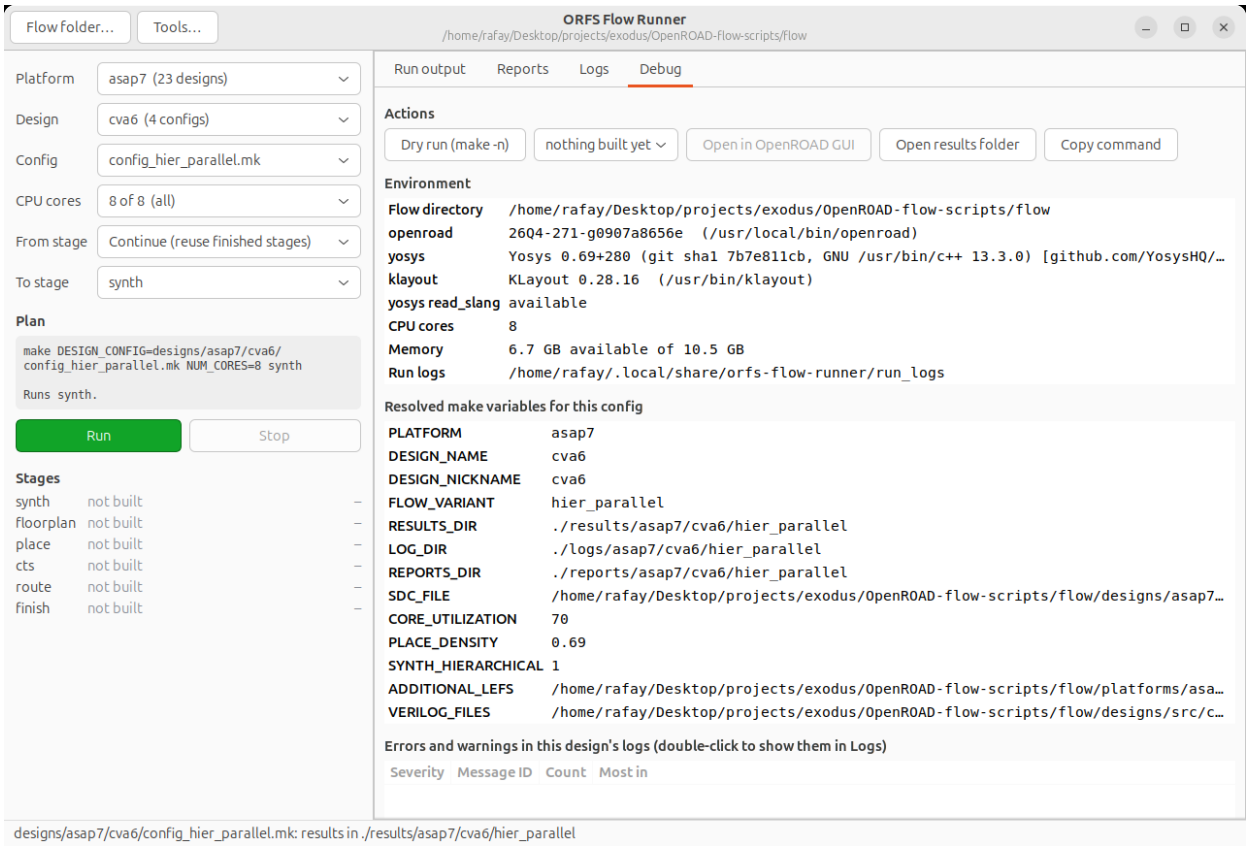


Figure 2. Debug → Environment after choosing the parallel Yosys: openroad, yosys and klayout found, read_slang available, logs in the per-user folder.

4. The window

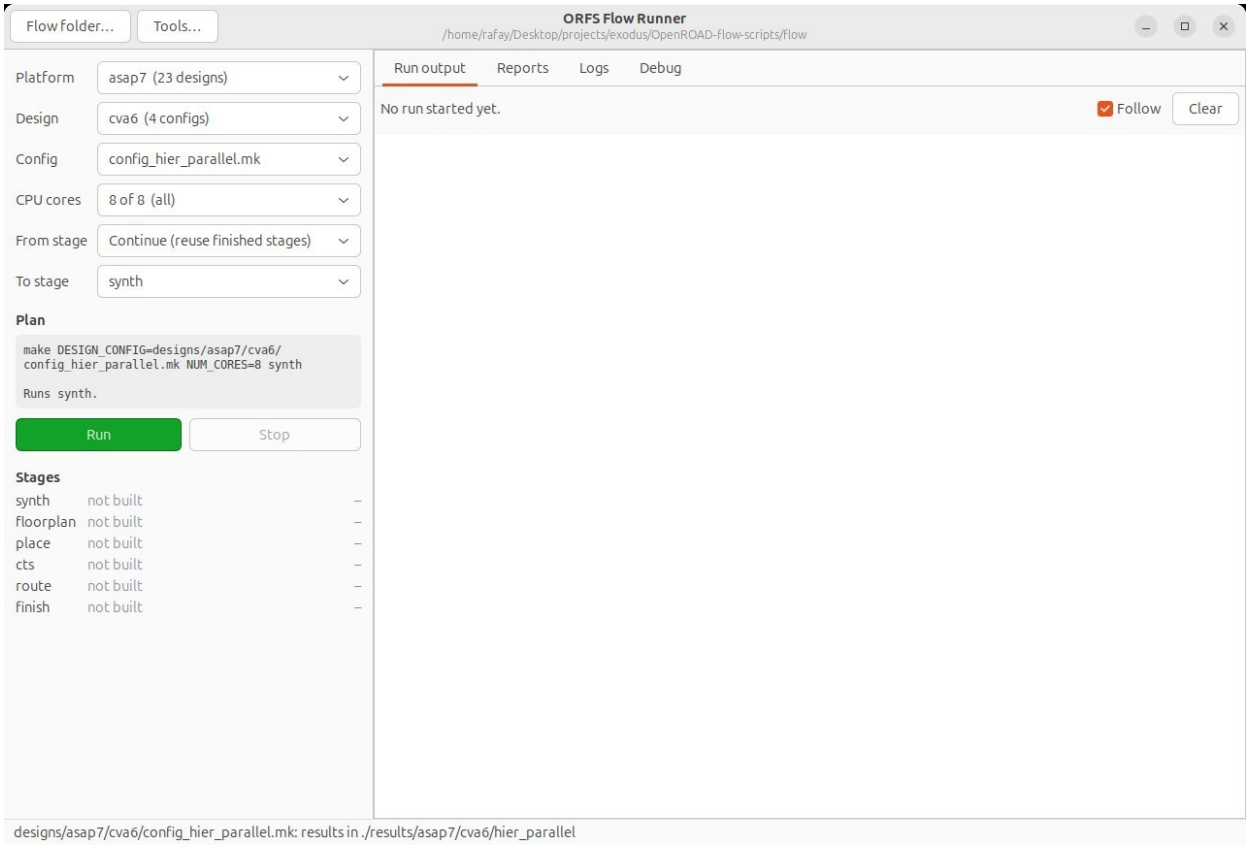


Figure 3. The window before a run: menus and plan on the left, output and results tabs on the right, status bar at the bottom.

Part	What it does
Title bar	Flow folder... and Tools...; the subtitle is the flow folder in use
Platform	every folder in flow/designs with designs; platforms whose PDK is missing are greyed out
Design, Config	the designs of that platform and their config*.mk files; the config sets DESIGN_CONFIG and with it the results folder
CPU cores	passed as NUM_CORES; fewer cores use less memory
From stage	Continue (reuse finished stages) or Re-run from a stage, which cleans that stage and every later one first
To stage	the last make target: synth, floorplan, place, cts, route or finish
Plan	the exact commands, the stages they run and anything they delete
Run, Stop	start the plan; Stop kills the running step and keeps finished stages
Stages	each stage as not built, running, partial, interrupted or done, with its runtime and peak memory
Tabs	Inputs, Run output, Reports, Logs, Debug (sections 5 to 7)

Part	What it does
Status bar	while running: elapsed time, current step, tool, memory used and free; after: the result and the log path

The app remembers the flow folder, the menu choices and the tools for the next start. Figure 3 is from version 1.0.1; from 1.1.0 an Inputs tab comes first (section 6).

5. Running a design

This walk-through synthesises CVA6 with hierarchical synthesis. Smaller designs such as nangate45/gcd work the same way and finish in minutes.

1. **Platform:** asap7. **Design:** cva6. **Config:** config_hier_parallel.mk (section 8 explains the A/B configs; config_hier.mk is the plain one).
2. **CPU cores:** 8 of 8. **From stage:** Continue. **To stage:** synth.
3. **Read the plan box.** It reads make DESIGN_CONFIG=designs/asap7/cva6/config_hier_parallel.mk NUM_CORES=8 synth and Runs synth.
4. **Press Run.** Run output streams the output: errors in red, warnings in amber, the run's own messages in blue. Untick Follow to scroll back without being pulled to the end.
5. **Watch Stages and the status bar.** synth shows running, and the status bar names the step (1_2_yosys), the tool and the memory in use.
6. **Wait for >> exit code 0.** The status bar then says Run finished after the elapsed time, with the path of the run log.

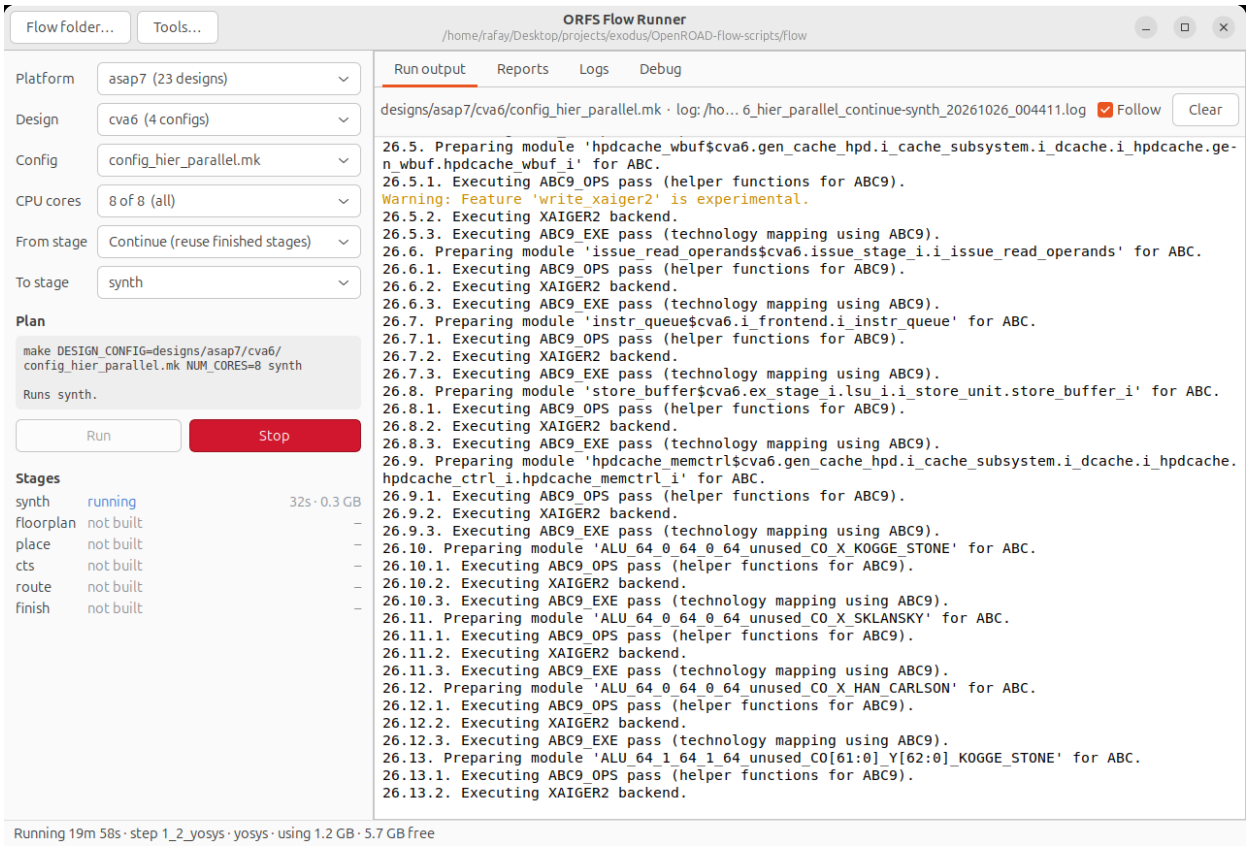


Figure 4. Twenty minutes into synthesis with the parallel Yosys: the step is 1_2_yosys and abc_new is preparing each module for ABC.

With stock Yosys the same point of the run looks different: abc_new maps one module at a time, so the output alternates Mapping module and ABC9 lines for each of the 93 modules (Figure 5).

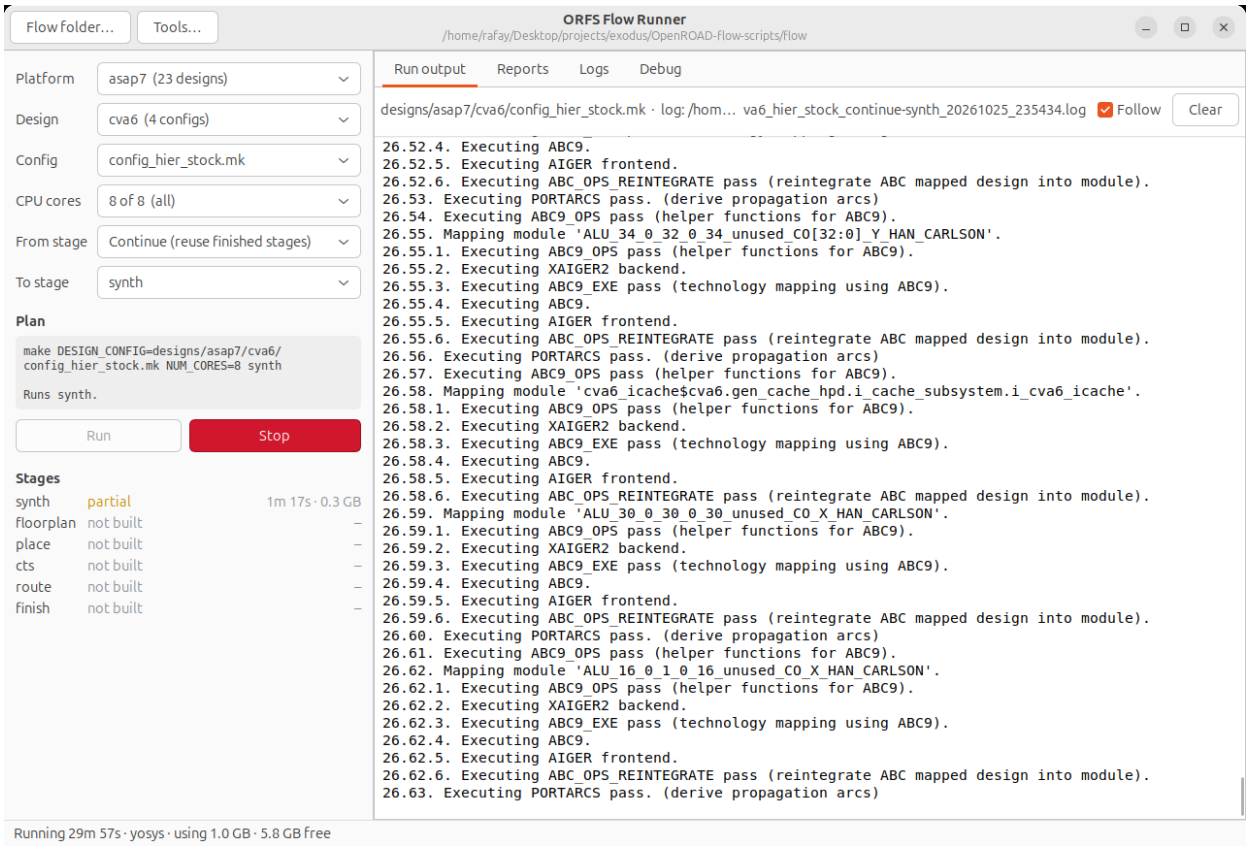


Figure 5. The same step with stock Yosys, 30 minutes in. Screenshot from app version 1.0.0, which showed synth as partial while it ran; 1.0.1 shows running.

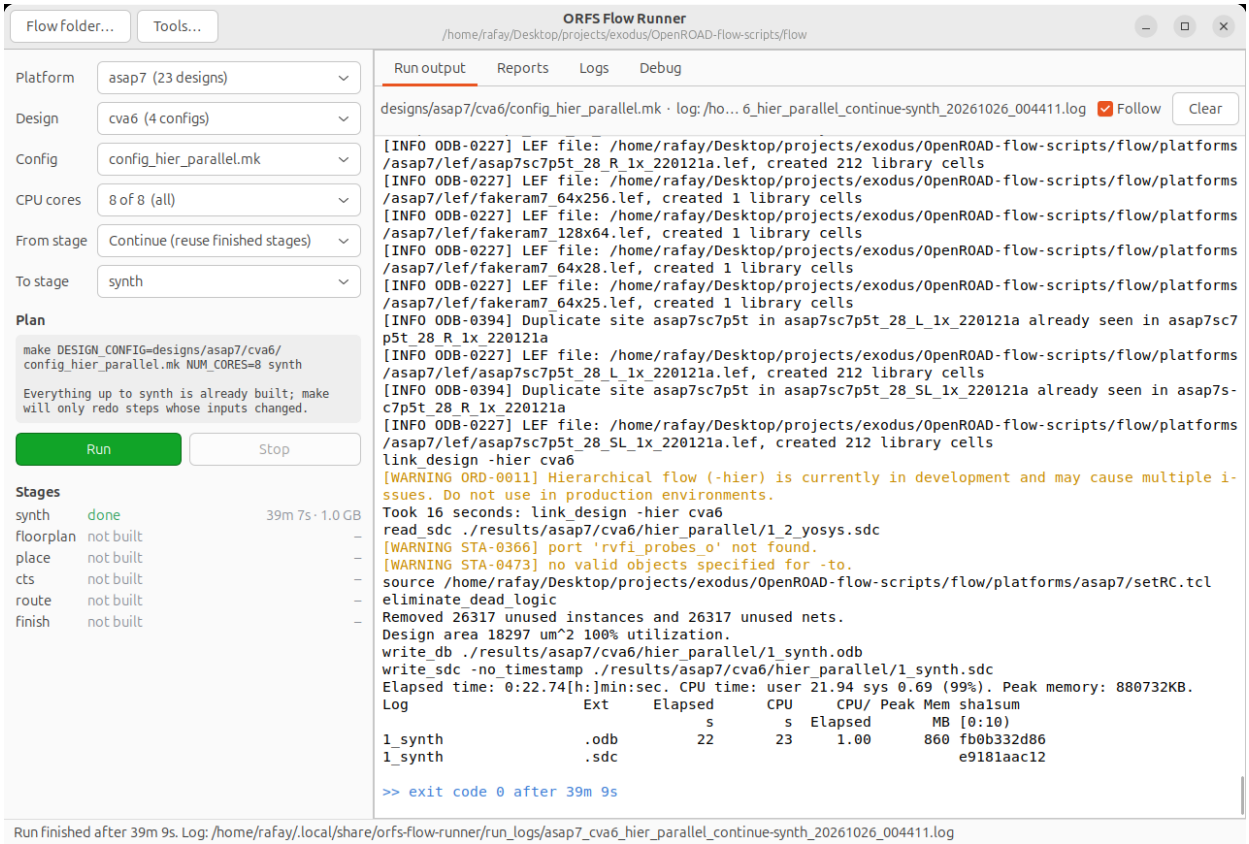


Figure 6. A finished synthesis run: synth done with its runtime and 1.0 GB peak, exit code 0, and the run log path in the status bar. From the first single-pair attempt; its timings are superseded by section 8.

To stop a run, press Stop and confirm. The running step is killed, its partial log stays as a .tmp.log and finished stages are kept; the stage then shows interrupted. Closing the window during a run asks first.

6. Setting the inputs

The Inputs tab edits everything a run reads, so a design can be set up and changed without a terminal. It has five pages: Clocks, Constraints, IO pins, Flow settings and Files; the line above them names the config being edited. Everything is saved into that config's own files (its config*.mk, SDC and Tcl files), so a run from the app and one from make use the same inputs. The screenshots in this chapter use CVA6 on nangate45 after synthesis, and ORFS's chi_async_bridge and ethmac designs on asap7.

6.1 Clocks

The app finds the top module (DESIGN_NAME) among VERILOG_FILES and reads its ports. An input counts as a clock if it is used with posedge or negedge, drives the clock pin of an instance, or is named like a clock; the line at the top says which and why. The clocks the SDC already defines are read by running the SDC in tclsh with the SDC commands replaced by stubs, so variables, expr and foreach work as they do in OpenROAD.

1. **Create a clock.** A clock port from the RTL with no clock yet is a row with an empty period: type the period to create the clock, and rename it if you like. Add clock... adds a clock on any input, or a virtual clock for input and output delays.
2. **Change a period.** For a clock defined in the file, the app edits the line the period comes from: the set clk_period line it uses, or the number in its create_clock line. Clocks that share a period variable change together.
3. **Relate the clocks.** Each pair of clocks on ports has a choice: timed together (one source), asynchronous (no fixed phase; paths between them are not timed and need synchronisers), logically exclusive (a mux picks one) or physically exclusive (they never exist at the same time).

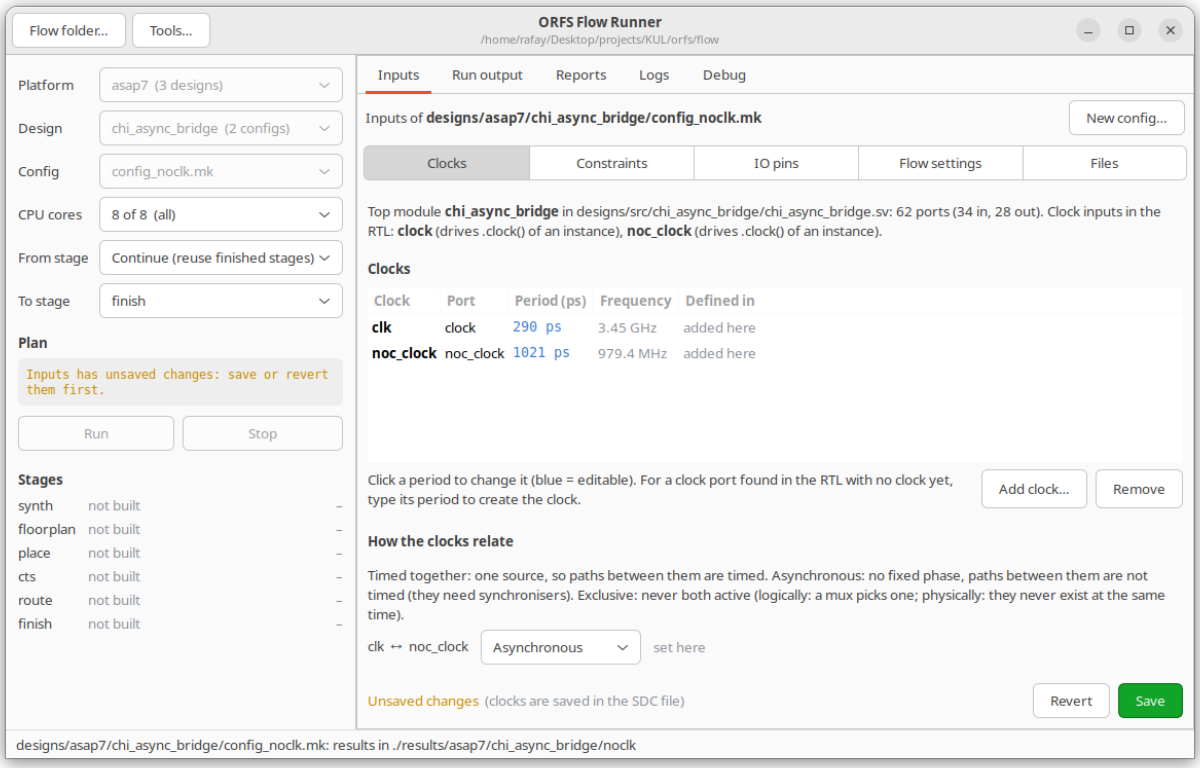


Figure 7. chi_async_bridge with an SDC that has no clocks: clock and noc_clock found in the RTL, periods typed in, and the two clocks made asynchronous.

What is set here is written to one marked block of the SDC (create_clock and set_clock_groups lines) after the file's own clock definitions; the rest of the file is not touched. Relations the file already sets are shown read-only.

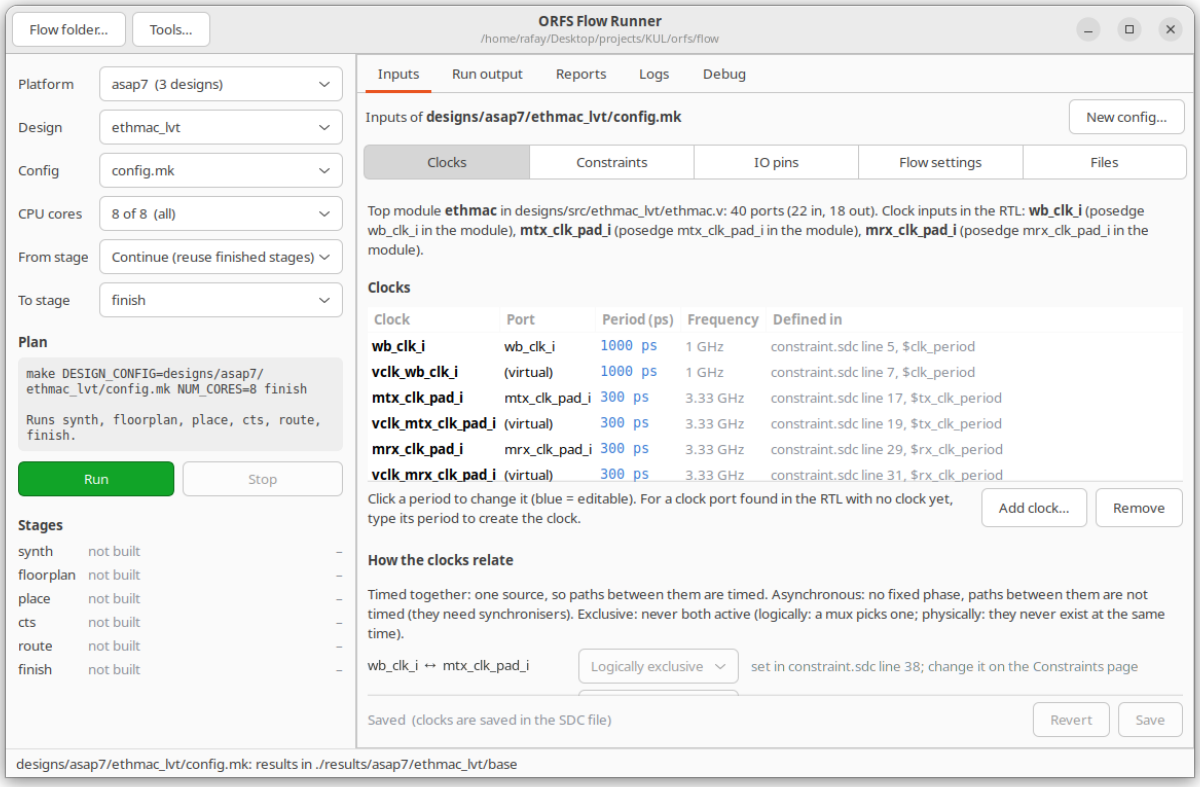


Figure 8. ethmac's six clocks read from its SDC, three real and three virtual, each period shown with the variable it comes from, and the logically exclusive groups the file sets.

6.2 Constraints

The SDC the config uses, as quick settings and as the whole file. If other configs use the same SDC, the page says so, and Use a separate copy for this config gives this config its own copy.

- **Quick settings:** a field for every value the SDC sets with set, and for a number in create_clock -period. A clock period shows its frequency (ps on asap7, ns elsewhere). A clock port is a list of the design's input ports, and a name that is not a port is shown in red. A value that no command uses is marked set but not used: CVA6's SDCs set input_delay and output_delay without applying them, so its inputs and outputs are not timed.
- **Input and output delays:** Add input and output delays asks which clock to use and adds set_input_delay on every input except the clocks and set_output_delay on every output. Delay for specific ports... adds a line for the ports you tick, optionally only some bits of a bus, which overrides the general delay for them.
- **Check with OpenROAD:** reads the SDC as it is in the editor, unsaved changes included, into OpenROAD with the synthesized design, as the flow does, and lists each clock with its period and the number of ports it is on, and every name OpenROAD cannot find.

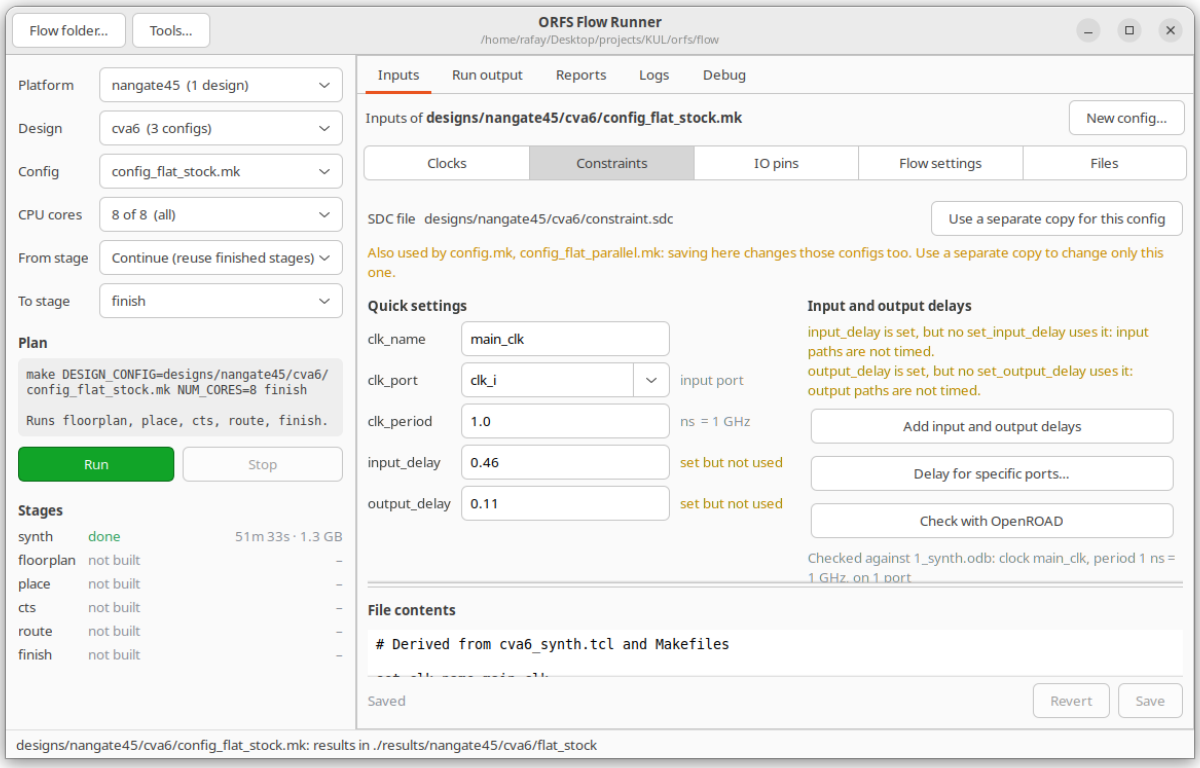


Figure 9. CVA6's SDC: clk_port is an input port, input_delay and output_delay are set but not used, and OpenROAD finds main_clk at 1 ns on one port.

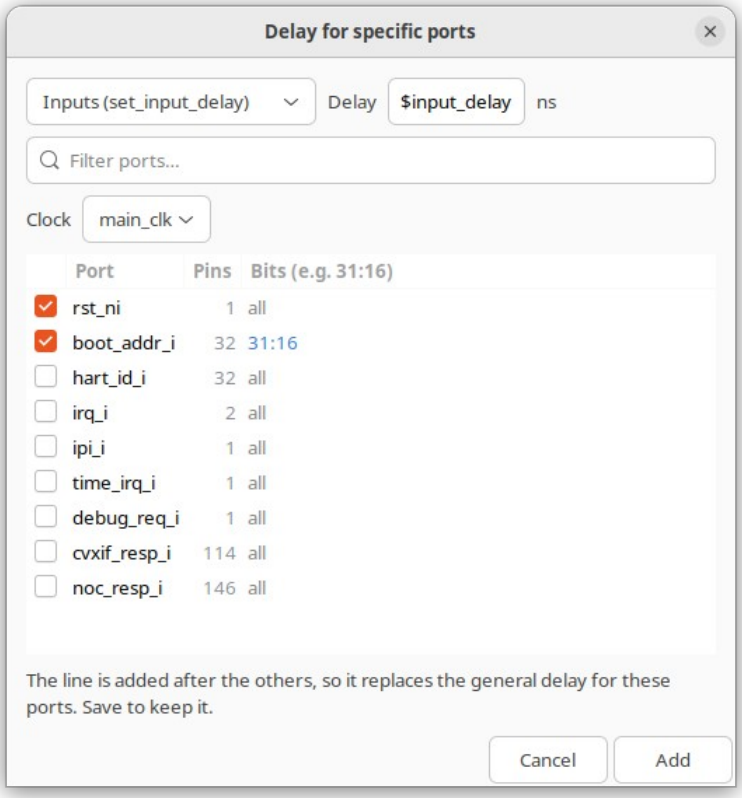


Figure 10. Delay for specific ports: rst_ni and bits 31:16 of boot_addr_i, on the clock main_clk.

6.3 IO pins

Where the pins go on the die edge, written as set_io_pin_constraint lines to the config's IO_CONSTRAINTS file, which the floorplan stage reads. If the config has none, saving creates io_constraints.tcl in the design folder and sets it.

1. **Read the table.** One row per port, with its number of pins: from the synthesized netlist, or from the RTL when the width is a plain number. A bus name stands for all its bits, as it does in OpenROAD.
2. **Place.** Select rows, choose an edge and a range in microns (* for the whole edge, or 0-200), and press Place. together keeps a bus's pins next to each other (-group); in order also keeps them in bit order (-order).
3. **Split a bus.** Split bus... turns a bus into two rows at a bit you choose, such as boot_addr_i[31:16] and boot_addr_i[15:0], so the parts can go on different edges; Join puts them back.
4. **Check.** Saving checks that bit ranges fit their bus and that no pin is placed twice. Check with OpenROAD looks up every pin name in the synthesized design: an unknown name would stop the floorplan (PPL-0061).

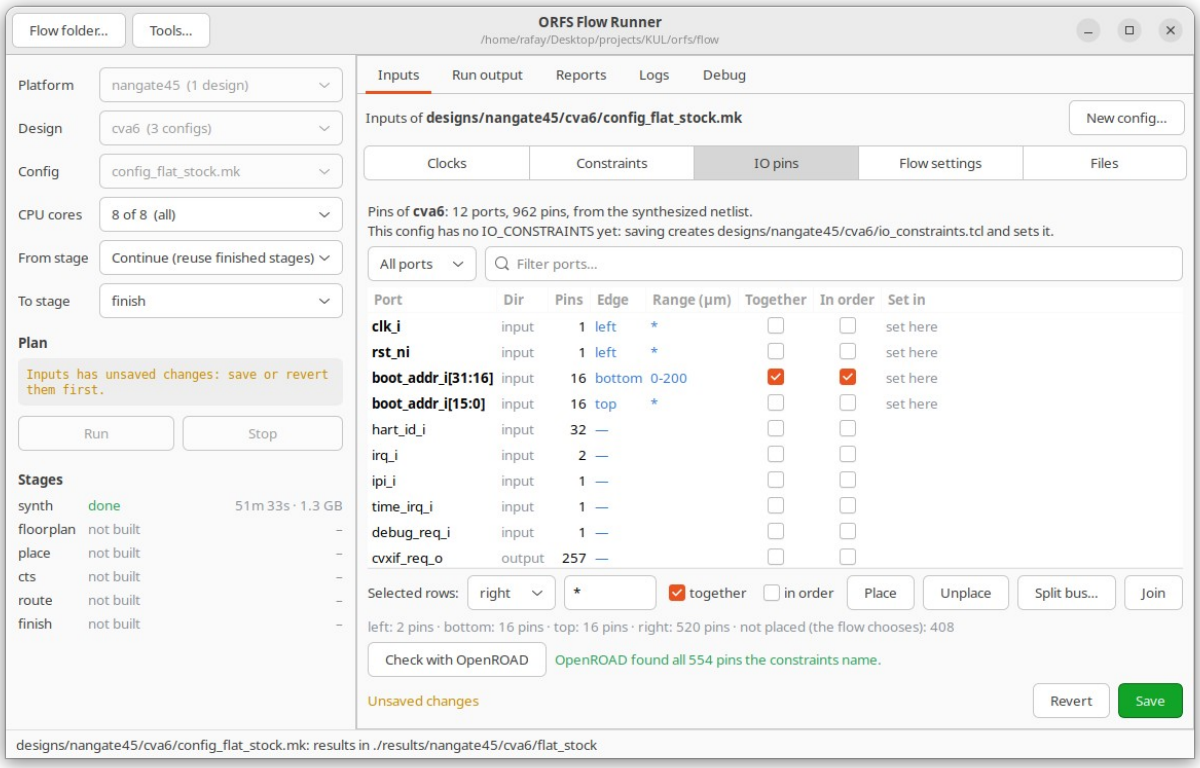


Figure 11. CVA6: clock and reset on the left, boot_addr_i split between the bottom (0-200 µm, together, in bit order) and the top, the NoC buses on the right; OpenROAD finds all 554 pins.

Constraints already in the file outside the page's block are shown read-only and kept. In a test on nangate45 gcd run through placement, every constrained pin was placed on its edge and a bus kept together in order came out in bit order.

6.4 Flow settings

Every ORFS variable from scripts/variables.json, with the value set in this config, the value the flow actually uses (make resolves it, including included configs and platform defaults), where that

value is set and the first stage that reads it. Hover a row for ORFS's description and default. The list opens on common settings: die and core area, utilization, aspect ratio, margins, the IO and macro placement files, PDN, placement density, timing repair and routing layers. Click a value to change it; an empty value removes it from this config.

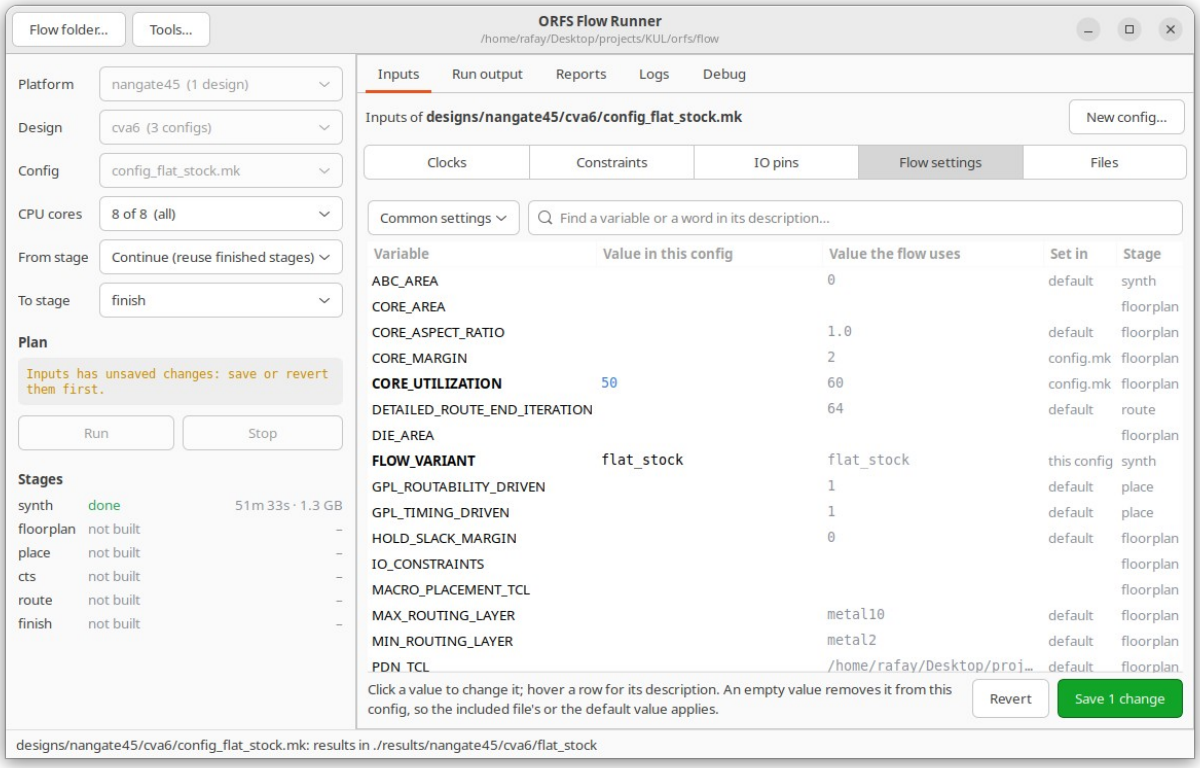


Figure 12. CORE_UTILIZATION changed to 50 in config_flat_stock.mk, not saved yet: the value the flow uses is still 60, from config.mk.

6.5 Files and new configs

Files edits any text file in the design folder and lists shared files the config uses, such as the platform's PDN script. New file... adds a file from a template: IO pin constraints and Macro placement list the design's ports and macros from the synthesized netlist, and set IO_CONSTRAINTS or MACRO_PLACEMENT_TCL. New config... (top right) makes config_<name>.mk, which includes the selected config and has its own results folder.

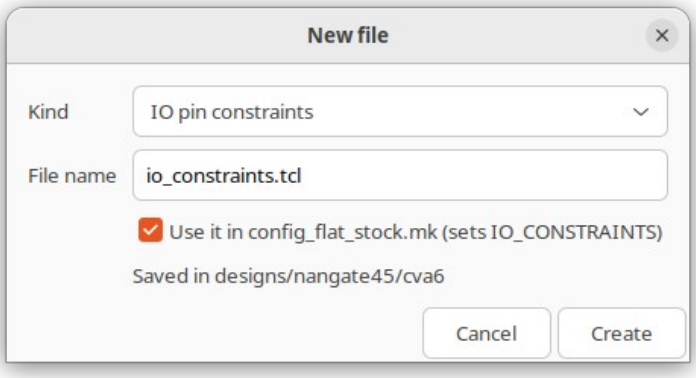


Figure 13. New file...: an IO pin constraints file, set as this config's IO_CONSTRAINTS.

6.6 Saving

- **Backup and undo:** every save keeps the old file in ~/.cache/orfs-flow-runner/backups/, and the bar that appears offers Undo save.
- **Checked by make:** make reads the config again after a save; if it no longer parses, the error is shown and Undo save puts the old files back.
- **From stage:** the app works out the first stage that reads what changed (synth for the SDC, floorplan for CORE_UTILIZATION or the IO pins). If that stage or a later one is built, From stage is set to re-run from it.
- **While editing:** Run and the Platform, Design and Config menus wait until unsaved changes are saved or reverted, and saving waits while a run is going.

7. Reading the results

7.1 Reports → QoR checks

QoR checks has pass/fail tiles for the latest stage that has metrics, then a table of every metric by stage. Red marks a failure: negative slack or a non-zero violation count. The numbers come from the ORFS metrics JSON files in logs/<platform>/<design>/<variant>/.

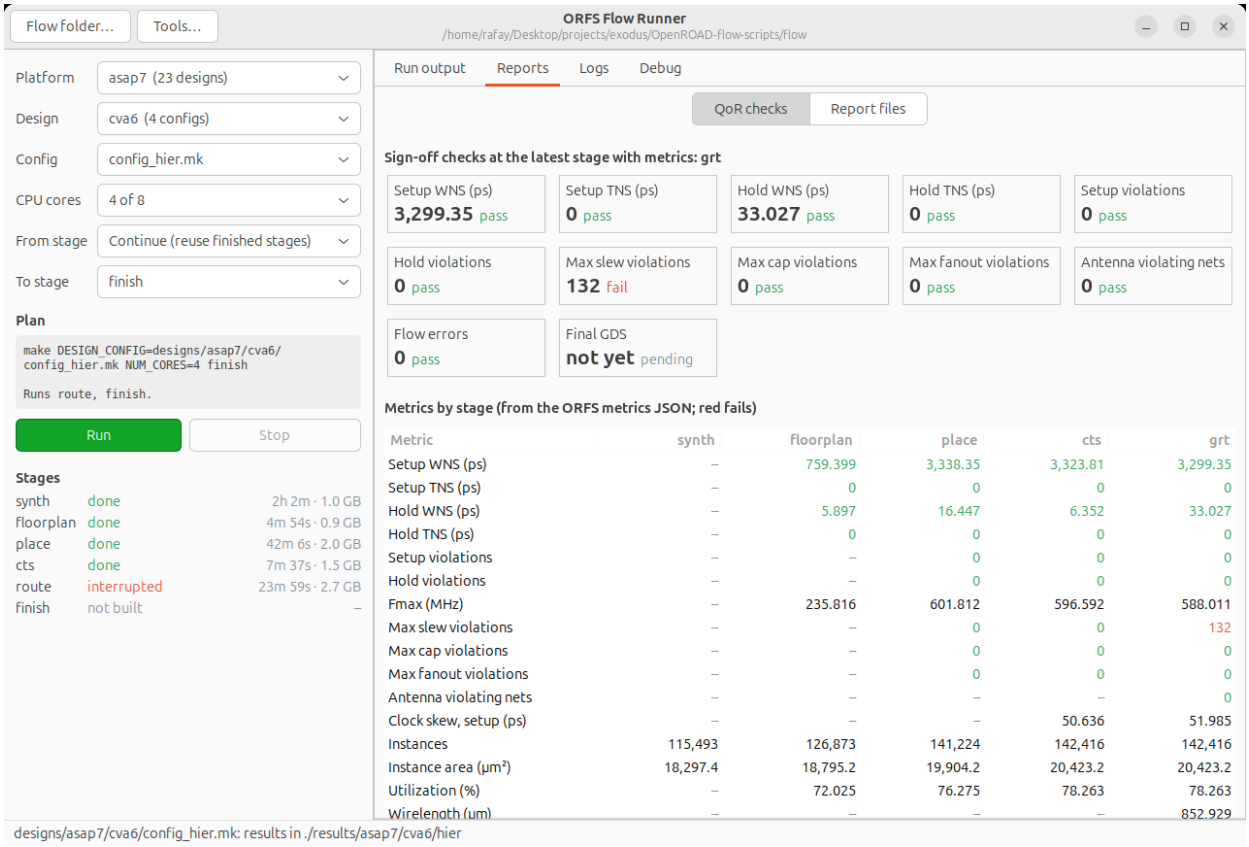


Figure 14. QoR checks for the earlier CVA6 hier run, which reached global routing before detailed routing was interrupted: timing passes, 132 max-slew violations fail.

After synthesis only, the table has one column: 115,493 instances and 18,297.4 μm² for CVA6, with 0 flow errors and 33 warnings. All four A/B runs show exactly these numbers.

7.2 Reports → Report files

Report files lists the design's reports folder. Pick a file to read it; type in the filter box to keep only matching lines, or tick Errors & warnings only.

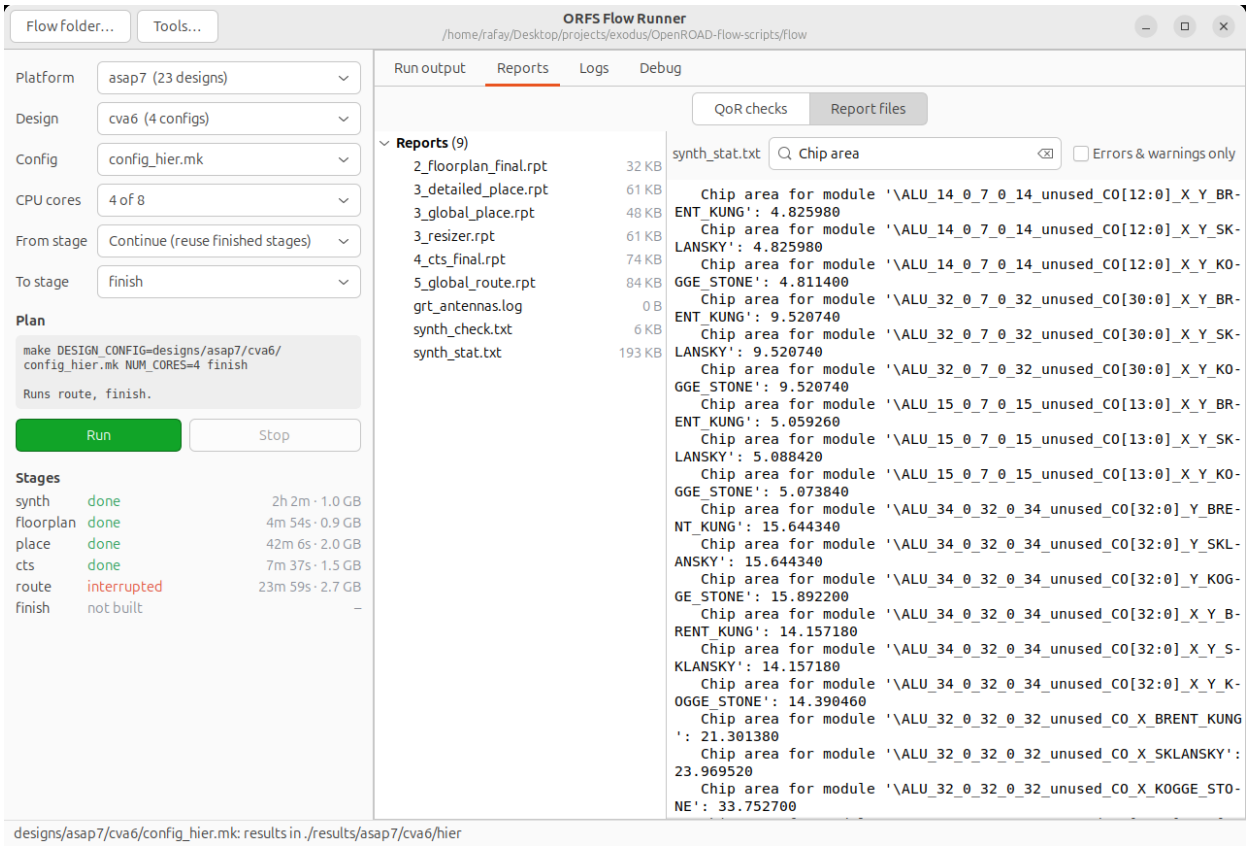


Figure 15. synth_stat.txt filtered to Chip area: the area of each synthesised module.

7.3 Logs

Logs lists the stage logs (one per step; a .tmp.log is a step that is running or was killed) and the run logs (one per run, newest first). The filter and Errors & warnings only work as in Report files. With Follow ticked, an open log that is still being written keeps scrolling to its end.

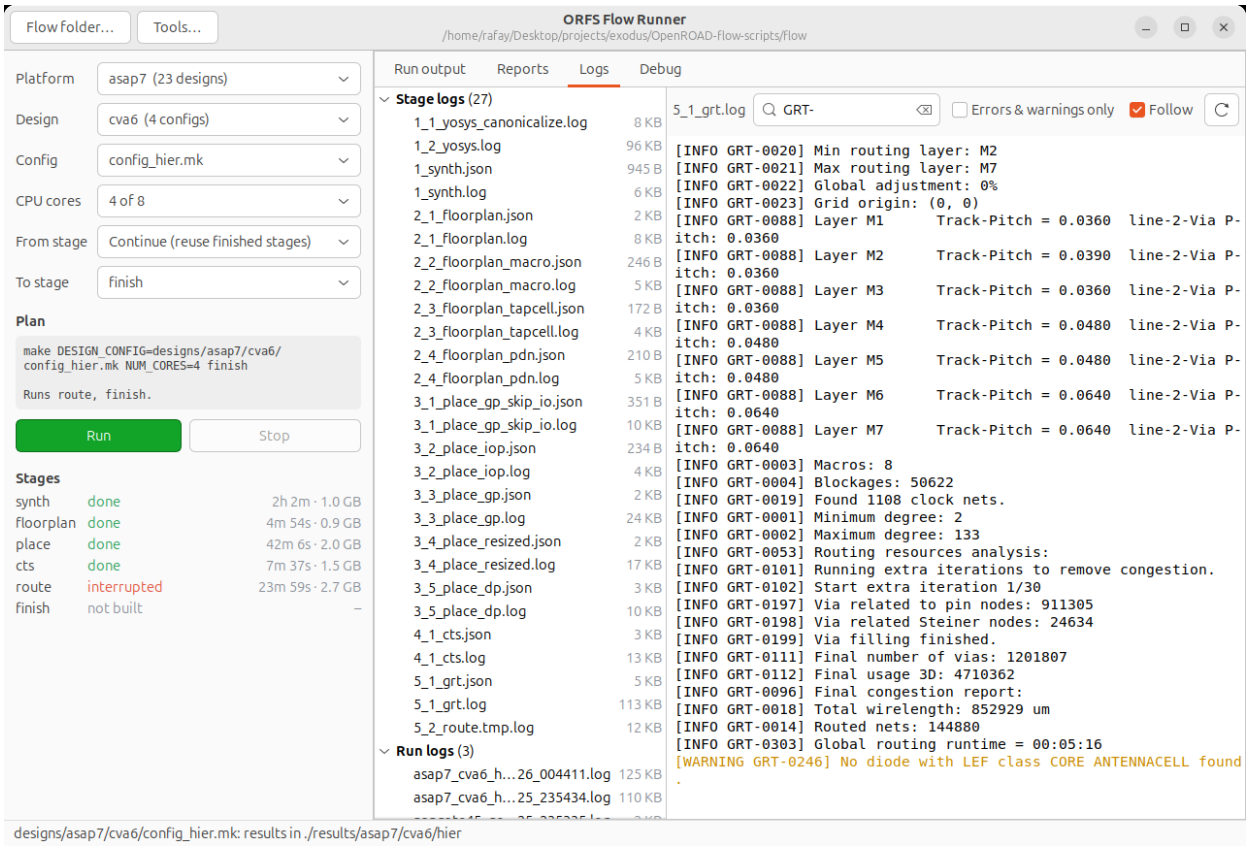


Figure 16. The global-route log filtered to GRT- messages; the run logs of the A/B runs are at the bottom of the list.

7.4 Debug

Debug has four parts. Actions: Dry run (make -n) shows what make would run for the plan; Open in OpenROAD GUI opens a built stage; Open results folder; Copy command copies the plan for a terminal. Environment (Figure 2). Resolved make variables: PLATFORM, FLOW_VARIANT, RESULTS_DIR, SDC_FILE, CORE_UTILIZATION and others as make print-<VAR> reports them. Errors and warnings: a count of every message ID in the design's logs, errors first; double-click one to open its log in Logs, filtered to that ID.

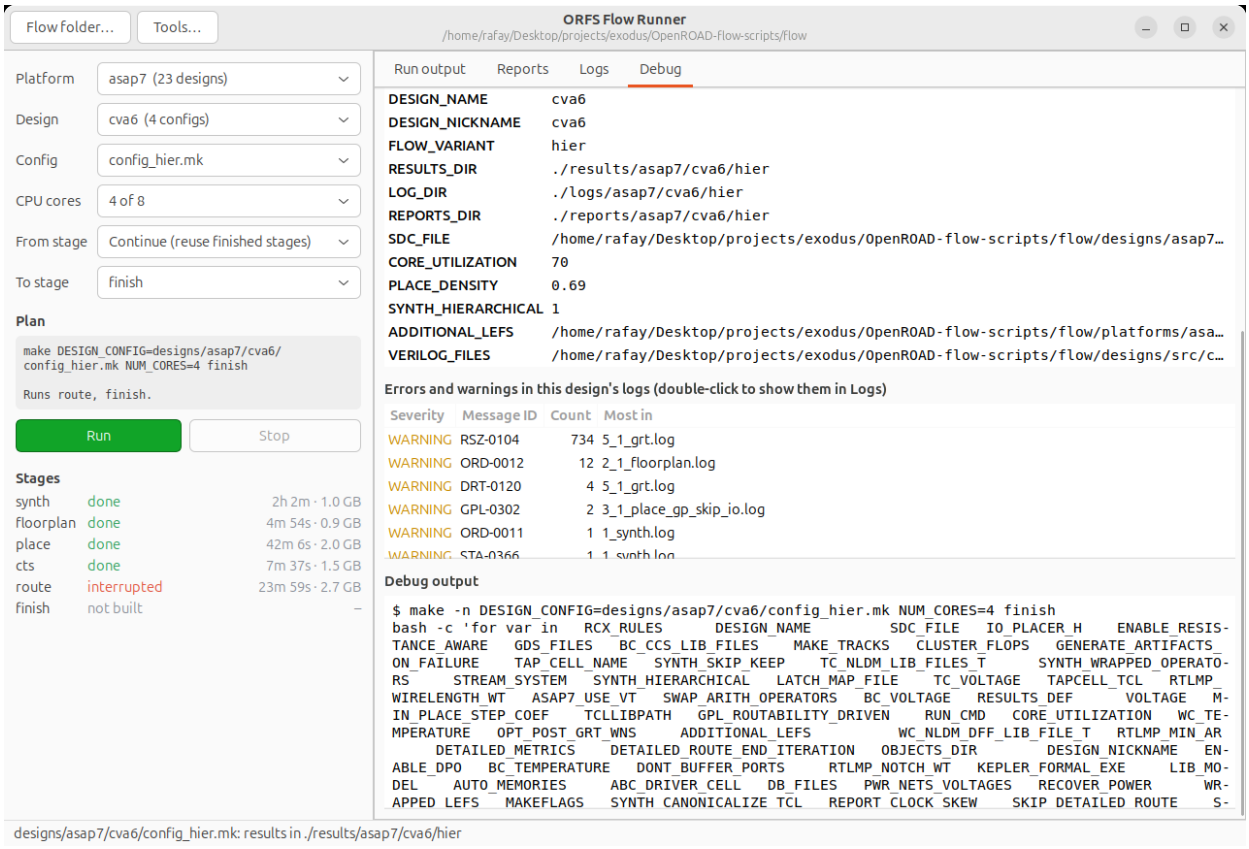


Figure 17. Debug scrolled down: the make variables, the warning IDs in the CVA6 logs (RSZ-0104 734 times, mostly in global routing) and a dry run.

8. Worked example: stock and parallel abc_new

8.1 What was compared

CVA6 synthesis spends most of its time in Yosys, and much of that in abc_new, which maps each of the 93 kept modules with ABC. The yosys/ submodule adds a parallel abc_new that maps independent modules at the same time. Runs from the app measure what that saves on a full ORFS synthesis.

	Stock	Parallel
Config	config_hier_stock.mk	config_hier_parallel.mk
Results folder	results/asap7/cva6/hier_stock	results/asap7/cva6/hier_parallel
Yosys (Tools...)	/usr/local/bin/yosys, 0.69+274	yosys/build/yosys, 0.69+280 (0.69+274 plus parallel abc_new)
Settings	NUM_CORES=8, From Continue, To synth	same

Both configs include config_hier.mk and change only FLOW_VARIANT, so the runs use identical settings and keep their results apart.

Method. The first attempt ran each build once, stock first, with a cold file cache: 47 min 55 s against 39 min 9 s. That is not a fair test, so it was repeated from scratch as four runs in the order

stock, parallel, parallel, stock (26 October 2026, 13:17 to 16:22). Before every run the variant's results, logs, reports and objects were deleted, the CVA6 sources, the asap7 platform files, both Yosys builds and OpenROAD were read into memory, and the machine idled for 30 seconds. Every run was started from the app with Yosys chosen in Tools…. The machine is a VirtualBox VM, so other load on its host can affect it.

```
# orfs/flow/designs/asap7/cva6/config_hier_parallel.mk
include $(dir $(lastword $(MAKEFILE_LIST)))config_hier.mk
export FLOW_VARIANT = hier_parallel
```

8.2 How to repeat it

1. Build Yosys from the yosys/ submodule (README, Step 2).
2. In the app choose asap7 / cva6 / config_hier_stock.mk, To stage synth, and set Yosys to the stock build in Tools…. Press Run.
3. When it finishes, choose config_hier_parallel.mk, set Yosys to yosys/build/yosys in Tools…, and press Run.
4. Repeat in the order stock, parallel, parallel, stock, choosing Re-run from synth so each run starts from scratch, and compare the Stages times and the end of each variant's 1_2_yosys.log (Took … seconds: abc_new, the time per pass and the CPU time).
5. Compare the CPU time of the Yosys step between runs as well: it does the same work every time, so a run that used much more CPU time ran on a slower machine.

8.3 Results

Run	Build	Whole synthesis	Yosys step	abc_new	Yosys CPU time
1	stock	2,562 s (42:42)	2,510 s	1,427 s	3,054 s
2	parallel	2,255 s (37:35)	2,198 s	1,105 s	3,076 s
3	parallel	2,873 s (47:53)	2,800 s	1,711 s	3,890 s
4	stock	3,159 s (52:39)	3,084 s	1,453 s	3,664 s

Comparison	Whole synthesis	Yosys step	abc_new
Runs 1 and 2: machine at normal speed (CPU time about 3,060 s)	1.14×	1.14×	1.29×
Runs 4 and 3: machine slower (CPU time 3,660 to 3,890 s)	1.10×	1.10×	0.85×
Mean of the two runs of each build (stock 2,860 s, parallel 2,564 s)	1.12×	1.12×	1.02×

Each factor is the stock time divided by the parallel time, so above 1 means the parallel build was faster. In all four runs the netlists are identical apart from the Yosys version comment, with 115,493 instances and 18,297.4 μm^2. Peak memory of Yosys was 1,009 and 1,017 MB for stock and 1,047 and 1,045 MB for parallel. The most yosys-abc processes seen at once (5 s samples) were 1 for stock and 4 for parallel.

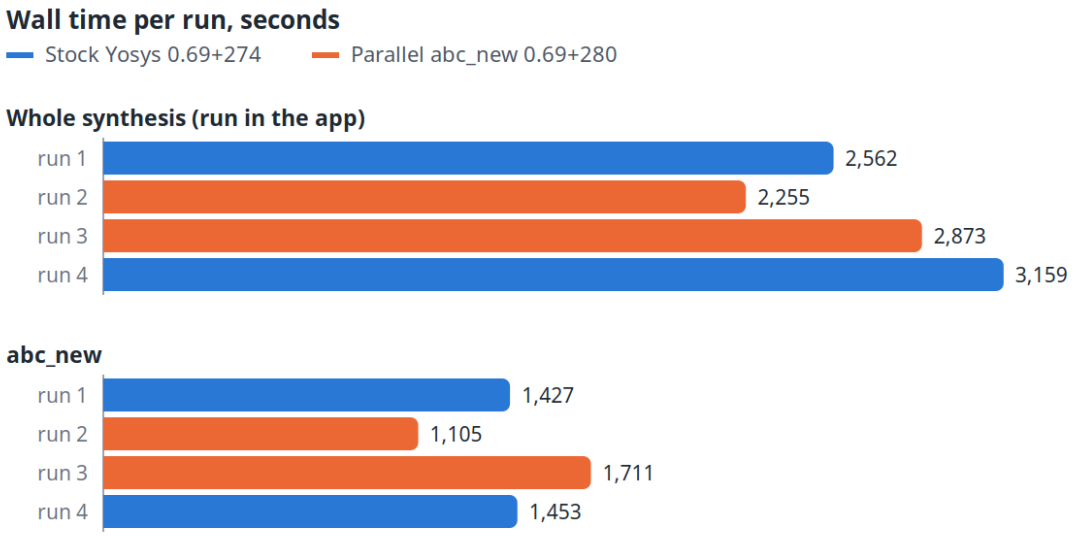


Figure 18. Wall time of each run: the whole synthesis and abc_new. Blue is stock, orange is parallel; bars are in run order.

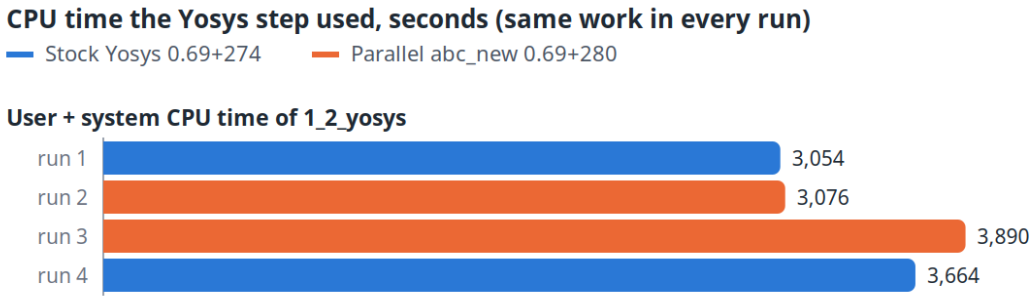


Figure 19. CPU time used by the Yosys step in each run. The work is the same every time, so runs 3 and 4 ran on a slower machine.

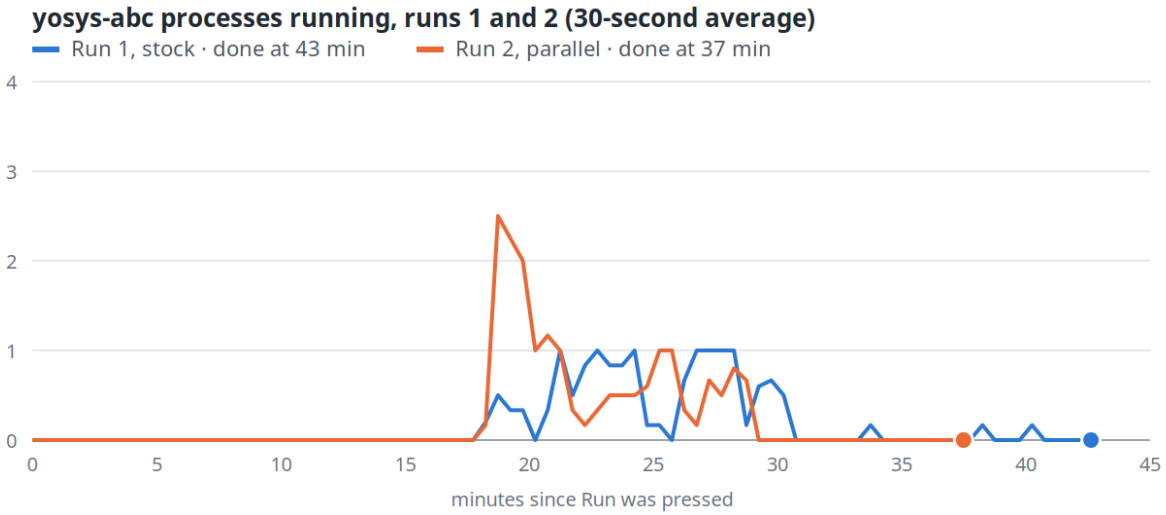


Figure 20. yosys-abc processes running in the cleanest pair, runs 1 and 2, averaged over 30 seconds. The parallel run reaches up to 4 processes; the stock run never has more than one.

8.4 What the numbers mean

- **Correctness:** the parallel build maps exactly as stock does: the four netlists are identical, so any speedup is free of changes to the result.

- **Speedup:** in the cleanest pair, runs 1 and 2, abc_new was 1.29× faster and the whole synthesis 1.14×, in line with the 1.2× to 1.5× measured by replaying abc_new alone in the change report.
- **The machine was not steady:** runs 3 and 4 used 20 to 27% more CPU time for the same work (3,890 s and 3,664 s against about 3,060 s). The cause was not established; a shared VirtualBox host is the likely one. The slowdown lasted through both later runs, so the order stock, parallel, parallel, stock does not cancel it.
- **Noise is as large as the effect:** in the slowed pair the parallel abc_new took 1,711 s against 1,453 s for stock (0.85×), while the whole synthesis was still 1.10× faster. Averaged, abc_new is 1.02× and the whole synthesis 1.12×. Two runs per build cannot give a confidence interval; a fair summary is abc_new between 0.85× and 1.29× and the whole synthesis about 1.1× on this machine.
- **Parallelism stays low:** at most 4 ABC processes ran at once. Yosys still does serial work around each ABC call; in the parallel logs write_xaiger2 alone takes 352 and 464 s over 186 calls. That serial work caps the saving.
- **Next:** repeat on a quiet machine, ideally not a VM, with more runs per build, and use the CPU time of the Yosys step to reject runs that were slowed down. The earlier 47 min 55 s against 39 min 9 s (1.22×) is replaced by these results.

9. Other tasks

9.1 Re-run from a stage

After a save in the Inputs tab, From stage is already set to the first stage the change affects. Otherwise choose Re-run from <stage> in From stage. The plan lists the clean commands first, and Run asks for confirmation with the stages whose results, logs and reports will be deleted, partial ones included. Later stages are cleaned too, because they were built from the old results.

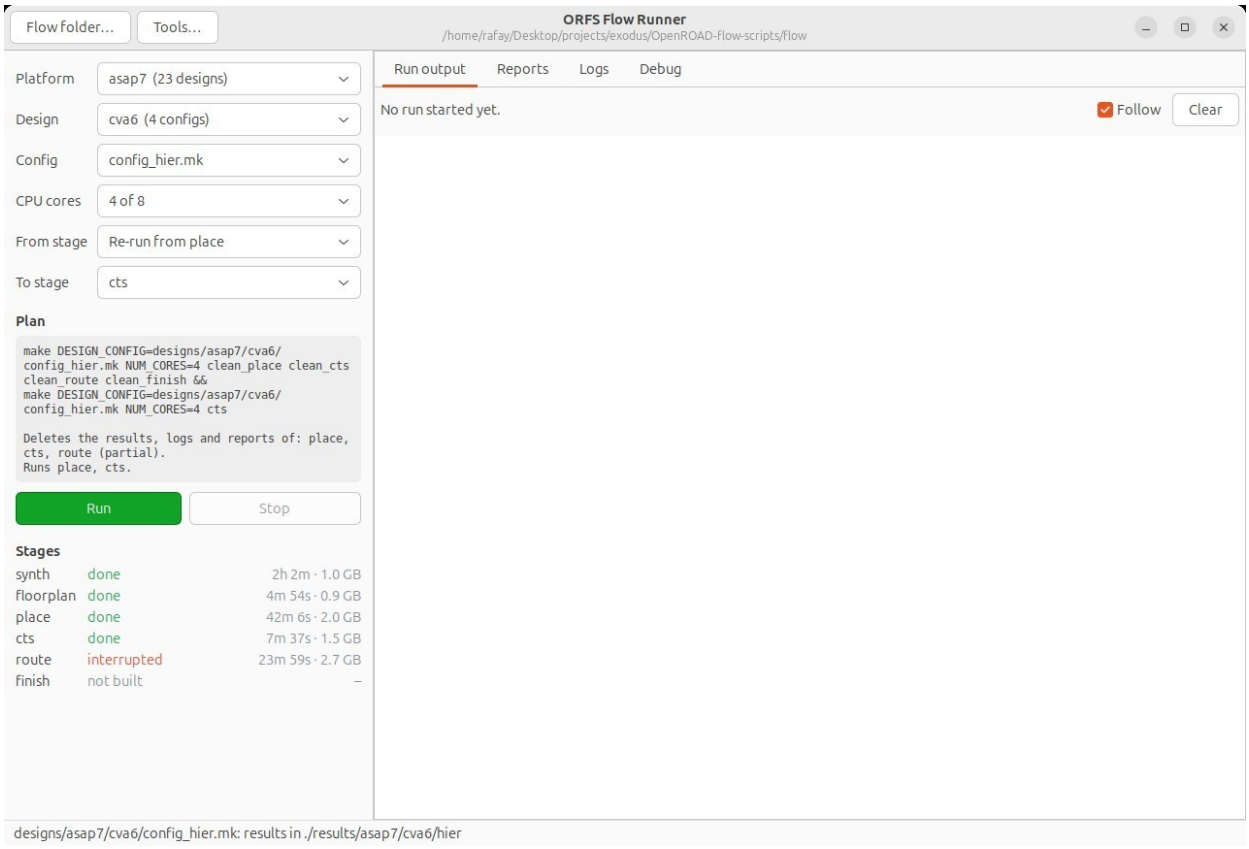


Figure 21. Re-run from place to cts on the earlier CVA6 run: the plan cleans place, cts and route, then runs place and cts.

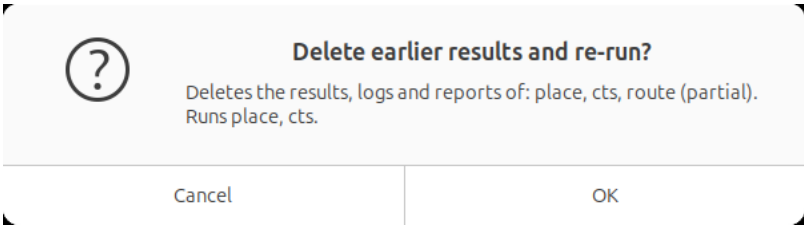


Figure 22. The confirmation before anything is deleted.

9.2 Runs that must outlive the app

Closing the app asks whether to stop its run. For a run that must survive logging out, use Debug → Copy command and start it with systemd-run --user as in the README's Running unattended section. The app still shows that run's results and logs, but not its live status.

9.3 Open a stage in the OpenROAD GUI

Debug → Actions: pick a built stage in the drop-down and click Open in OpenROAD GUI. Its output goes to ~/.cache/orfs-flow-runner/openroad_gui.log.

10. Troubleshooting

Problem	What to do
No module named gi at start	sudo apt install python3-gi gir1.2-gtk-3.0, and use the system python3, not a venv

Problem	What to do
A run fails at once with error 127	OpenROAD or Yosys was not found: check Debug → Environment and choose the program with Tools...
The Platform menu is empty	the folder is not an ORFS flow/ folder: use Flow folder...
A stage shows interrupted	the step was killed, usually for lack of memory; look for oom-kill in journalctl, use fewer cores, then run again with Continue
read_slang missing (CVA6)	build Yosys with the slang frontend (README, Step 1 and 2)
Make variables show make error	the config does not parse; the message is in Debug output
Run and the config menus are greyed out	the Inputs tab has unsaved changes: save or revert them
Clocks says tclsh was not found	sudo apt install tcl; the .deb installs it
The floorplan stops with PPL-0061	a pin name in IO_CONSTRAINTS does not exist: open Inputs → IO pins and press Check with OpenROAD
The installed app does not appear in the menu	an older gui/install_launcher.sh entry hides it: run gui/install_launcher.sh --uninstall

Limits: one run at a time per window; only runs started from the app show as running; viewers show the last 4 MB of a log, and the full output is always in the run log.

11. Files

Path	What it is
gui/orfs_app.py	the app
gui/inputs_tab.py	the Inputs tab
gui/sdc_clocks.py, gui/io_pins.py	read clock ports from the RTL and clocks from an SDC; read and write IO pin constraints
gui/flow_gui.py	the CVA6 web console; the app reuses its metric definitions
gui/packaging/build_deb.sh	builds the .deb; orfs-flow-runner.desktop and .svg are its menu entry and icon
gui/install_launcher.sh	menu and Desktop entry for running from a checkout, without the package
orfs/flow/designs/asap7/cva6/config_hier_{stock,parallel}.mk	the A/B configs of section 8
~/.config/orfs-flow-runner/settings.json	flow folder, menu choices and tools
~/.local/share/orfs-flow-runner/run_logs/	one log per run: <platform>_<design>_<variant>_<from>-<to>_<date>_<time>.log

Path	What it is
~/.cache/orfs-flow-runner/	OpenROAD GUI log, and backups/ with every file the Inputs tab replaced; safe to delete
<flow>/results|logs|reports/<platform>/<design>/<variant>/	ORFS results (one .odb per stage, 6_final.gds), step logs with metrics JSON, and reports